BÁO CÁO BÀI THỰC HÀNH

LẬP TRÌNH ỨNG DỤNG DI ĐỘNG

ỨNG DỤNG DANH SÁCH SẢN PHẨM

Kotlin – Android – JSON Assets

Sinh viên thực hiện: ĐÔNG ĐỨC NGUYÊN

Mã sinh viên: B23DCVT311

Nội dung: Danh sách, lọc, tìm kiếm và chi tiết sản phẩm

Ngày 02 tháng 10 năm 2026

Tóm tắt

Bài thực hành xây dựng một ứng dụng Android bằng Kotlin để trình bày danh mục sản phẩm. Dữ liệu gồm 30 sản phẩm thuộc 6 danh mục, được đọc trực tiếp từ tệp `app/src/main/assets/database.json`; ảnh chụp sản phẩm đại diện được đóng gói cùng ứng dụng. Màn hình thứ nhất hiển thị danh sách bằng `ListView`, `ArrayList<Product>` và `ArrayAdapter<Product>`; hỗ trợ lọc theo danh mục và tìm kiếm theo tên hoặc mã. Màn hình thứ hai hiển thị thông tin chi tiết của sản phẩm được chọn. Báo cáo trình bày yêu cầu, thiết kế, mã nguồn chính, ảnh chụp màn hình và kết quả kiểm tra.

Mục lục

1 Tổng quan bài thực hành **1**

1.1 Bối cảnh và mục tiêu 1

1.2 Yêu cầu chức năng 1

1.3 Phạm vi triển khai 1

2 Phân tích và thiết kế **1**

2.1 Các tác nhân và luồng sử dụng 1

2.2 Kiến trúc dữ liệu và giao diện 2

2.3 Cấu trúc dự án 2

3 Thiết kế dữ liệu JSON **3**

3.1 Cấu trúc một sản phẩm 3

3.2 Phân bố dữ liệu 3

3.3 Mô hình Kotlin 3

4 Triển khai chức năng **4**

4.1 Đọc trực tiếp tệp JSON 4

4.2 Tạo ArrayList, ArrayAdapter và setAdapter 5

4.3 Lọc sản phẩm bằng Spinner 6

4.4 Tìm kiếm độc lập với bộ lọc 7

4.5 Mở Activity chi tiết 8

4.6 Đọc ảnh từ assets và hiển thị giao diện 8

5 Ảnh chụp màn hình và đối chiếu chức năng **9**

5.1 Danh sách ban đầu và lọc danh mục 10

5.2 Tìm kiếm có kết quả và không có kết quả 11
5.3 Màn hình chi tiết sản phẩm 12

6 Kiểm thử và đánh giá **12**
6.1 Môi trường 12
6.2 Các trường hợp kiểm tra 13
6.3 Điểm đạt được và giới hạn 13

7 Kết luận **13**

A Hướng dẫn biên dịch và mở báo cáo **14**
A.1 Ứng dụng Android 14
A.2 Báo cáo LaTeX 14

1 Tổng quan bài thực hành

1.1 Bối cảnh và mục tiêu

Mục tiêu của bài thực hành là kết hợp đọc dữ liệu JSON cục bộ với các thành phần giao diện Android cơ bản. Ứng dụng cần đưa dữ liệu từ tệp vào mô hình Kotlin, chuyển mô hình thành các hàng của danh sách, xử lý thao tác lọc và tìm kiếm, rồi mở một Activity khác để xem chi tiết. Đây là quy trình đầy đủ từ dữ liệu đến giao diện và điều hướng.

1.2 Yêu cầu chức năng

1. Tạo tệp `database.json` chứa khoảng 30 bản ghi. Mỗi bản ghi có `code`, `name`, `price`, `category`, `image` và `description`.
2. Tạo `ArrayList<Product>`, lớp `ArrayAdapter<Product>` và gán adapter cho `ListView` bằng `setAdapter`.
3. Trên màn hình danh sách, hiển thị tên và mã sinh viên, tiêu đề "Danh sách sản phẩm", Spinner lọc danh mục, ô nhập và nút tìm kiếm.
4. Tìm kiếm phải xét toàn bộ dữ liệu, không bị giới hạn bởi danh mục đang chọn. Khi tìm kiếm, Spinner trở về "Tất cả".
5. Khi nhấn một sản phẩm, mở màn hình chi tiết, hiển thị tên, ảnh và mô tả. Cả hai Activity đều có phần thông tin sinh viên ở phía trên.

1.3 Phạm vi triển khai

Ứng dụng có hai Activity: `MainActivity` và `ProductDetailActivity`. Dữ liệu và ảnh đặt trong thư mục `assets`, vì vậy các chức năng nêu trên chạy không cần kết nối Internet. Giá được biểu diễn bằng số nguyên đồng Việt Nam và định dạng theo vùng ngôn ngữ Việt Nam khi hiển thị.

2 Phân tích và thiết kế

2.1 Các tác nhân và luồng sử dụng

Người dùng mở ứng dụng để xem toàn bộ sản phẩm. Sau đó người dùng có thể chọn danh mục, nhập tên hoặc mã để tìm, chọn một hàng để xem chi tiết và trở lại danh sách. Bảng 1 tóm tắt đầu vào và kết quả mong đợi.

Bảng 1: Các chức năng chính

Chức năng	Đầu vào/thao tác	Kết quả
Xem danh sách	Mở ứng dụng	Hiển thị 30 sản phẩm, ảnh, tên, danh mục và giá.
Lọc danh mục	Chọn một mục trong Spinner	Chỉ hiện sản phẩm thuộc danh mục đó; "Tất cả" hiện toàn bộ.
Tìm kiếm	Nhập tên hoặc mã rồi nhấn Tìm kiếm	Tìm trong toàn bộ 30 sản phẩm; Spinner trở về "Tất cả".
Xem chi tiết	Nhấn vào một sản phẩm	Mở Activity thứ hai với đúng tên, ảnh, mô tả, giá và mã.
Không có kết quả	Nhập chuỗi không khớp	Hiển thị số lượng 0 và thông báo rỗng.

2.2 Kiến trúc dữ liệu và giao diện

Ứng dụng dùng một luồng dữ liệu đơn giản, phù hợp với quy mô bài thực hành:

`database.json` và `assets/images`

↓

`ProductRepository.load()` ⟶ `ArrayList<Product>`

↓

`ProductAdapter` ⟶ `ListView` trong `MainActivity`

↓ nhấn một hàng, truyền `product_code`

`ProductDetailActivity` ⟶ tên, ảnh, mô tả và thông tin bổ sung

Tệp JSON là nguồn dữ liệu sản phẩm. `ProductRepository` chịu trách nhiệm đọc và chuyển từng đối tượng JSON thành `Product`. `ProductAdapter` nhận danh sách đang hiển thị và gắn dữ liệu vào bố cục của từng hàng. `ProductImages` xử lý đường dẫn ảnh cục bộ và dùng bộ đệm bitmap. Hai Activity đảm nhiệm giao diện và tương tác của từng màn hình.

2.3 Cấu trúc dự án

Bảng 2: Những tệp quan trọng

Tệp/thư mục	Vai trò
`assets/database.json`	Dữ liệu 30 sản phẩm.
`assets/images/`	30 ảnh PNG, mỗi sản phẩm một ảnh.
`Product.kt`	Mô hình dữ liệu sản phẩm.
`ProductRepository.kt`	Đọc JSON, tạo danh sách và định dạng giá.
`ProductAdapter.kt`	Hiển thị một sản phẩm trong `ListView`.
`ProductImages.kt`	Giải mã ảnh từ assets, quản lý bộ đệm.
`MainActivity.kt`	Danh sách, lọc, tìm kiếm và điều hướng.
`ProductDetailActivity.kt`	Hiển thị chi tiết sản phẩm.
`res/layout/`	XML bố cục của hai màn hình và một hàng sản phẩm.

3 Thiết kế dữ liệu JSON

3.1 Cấu trúc một sản phẩm

Mảng JSON có 30 phần tử. Mỗi phần tử có sáu thuộc tính bắt buộc. Thuộc tính `price` là số nguyên, không lưu ký hiệu tiền tệ trong dữ liệu. Thuộc tính `image` chứa URL cục bộ dạng `file:///android_asset/images/...`. Ví dụ sau là một bản ghi thực tế được trích từ tệp dữ liệu:

Đoạn mã: Bản ghi sản phẩm `SP001` trong `database.json`

```
{
  "code": "SP001",
  "name": "Nova X1 128GB",
  "price": 6990000,
  "category": "Điện thoại",
  "image": "file:///android_asset/images/SP001.png",
  "description": "Điện thoại màn hình 6,5 inch sắc nét, bộ nhớ 128GB và pin
  ↪  5.000mAh. Phù hợp cho học tập, chụp ảnh và giải trí mỗi ngày."
}
```

3.2 Phân bố dữ liệu

Các mã được tạo liên tiếp từ `SP001` đến `SP030`. Sáu danh mục có mỗi danh mục năm sản phẩm. Bảng 3 mô tả phân bố:

Bảng 3: Số lượng sản phẩm theo danh mục

Danh mục	Số lượng	Khoảng mã
Điện thoại	5	SP001–SP005
Máy tính	5	SP006–SP010
Âm thanh	5	SP011–SP015
Phụ kiện	5	SP016–SP020
Gia dụng	5	SP021–SP025
Thời trang	5	SP026–SP030
Tổng cộng	**30**	**SP001–SP030**

3.3 Mô hình Kotlin

Lớp dữ liệu giữ đúng sáu trường trong JSON. Kiểu `Long` cho giá tránh sai lệch do số thực và đủ để biểu diễn các mức giá của bài thực hành.

Đoạn mã: `Product.kt`

```
data class Product(
    val code: String,
    val name: String,
    val price: Long,
    val category: String,
    val image: String,
    val description: String
)
```

4 Triển khai chức năng

4.1 Đọc trực tiếp tệp JSON

Trong `ProductRepository.load()`, lệnh `context.assets.open("database.json")` đọc tệp được đóng gói trong APK. `JSONArray` duyệt từng phần tử, `JSONObject` lấy từng trường và thêm một đối tượng `Product` vào `ArrayList<Product>`. Dữ liệu trên màn hình vì thế xuất phát trực tiếp từ tệp JSON, không khai báo cứng danh sách sản phẩm trong Activity.

Đoạn mã: Phần cốt lõi của `ProductRepository.kt`

```
fun load(context: Context): ArrayList<Product> {
    val json = context.assets.open("database.json")
        .bufferedReader(Charsets.UTF_8).use { it.readText() }
    val records = JSONArray(json)
    val products = ArrayList<Product>(records.length())
    for (index in 0 until records.length()) {
        val item = records.getJSONObject(index)
        products.add(
            Product(
                code = item.getString("code"),
                name = item.getString("name"),
                price = item.getLong("price"),
                category = item.getString("category"),
                image = item.getString("image"),
                description = item.getString("description")
            )
        )
    }
    return products
}
```

Giá được định dạng riêng bằng `NumberFormat.getCurrencyInstance` với `Locale("vi", "VN")`. Ví dụ, giá trị `6990000` được hiển thị trên giao diện thành `6.990.000 đ`.

4.2 Tạo ArrayList, ArrayAdapter và setAdapter

`MainActivity` quản lý hai danh sách. `allProducts` giữ dữ liệu gốc; `visibleProducts` giữ kết quả đang hiển thị. Việc tách hai danh sách giúp lọc hoặc tìm kiếm mà không làm mất dữ liệu gốc. `ProductAdapter` nhận danh sách đang hiển thị. Trong lớp này, `getView()` gán tên, danh mục, giá và ảnh cho từng hàng.

Đoạn mã: Khởi tạo danh sách và gán adapter trong `MainActivity.kt`

```
private val allProducts = ArrayList<Product>()
private val visibleProducts = ArrayList<Product>()
private lateinit var productAdapter: ProductAdapter

allProducts.addAll(ProductRepository.load(this))
productAdapter = ProductAdapter(this, visibleProducts)
productList.adapter = productAdapter
productList.emptyView = findViewById(R.id.emptyMessage)
```

Dòng `productList.adapter = productAdapter` là cách viết thuộc tính Kotlin tương đương lời gọi `productList.setAdapter(productAdapter)` của Android. Phương thức `showProducts()` xóa dữ liệu cũ, thêm danh sách kết quả, gọi `notifyDataSetChanged()` để `ListView` vẽ lại và cập nhật nhãn số lượng.

Đoạn mã: Cập nhật danh sách đang hiển thị

```
private fun showProducts(products: List<Product>) {
    visibleProducts.clear()
    visibleProducts.addAll(products)
    productAdapter.notifyDataSetChanged()
    resultCount.text = "${visibleProducts.size} sản phẩm"
}
```

Đoạn mã: Phương thức `getView()` của `ProductAdapter.kt`

```
class ProductAdapter(
    context: Context, products: ArrayList<Product>
) : ArrayAdapter<Product>(context, 0, products) {
private val inflater = LayoutInflater.from(context)

override fun getView(
```

```
    position: Int, convertView: View?, parent: ViewGroup
): View {
    val view = convertView
        ?: inflater.inflate(R.layout.item_product, parent, false)
    val product = getItem(position) ?: return view
    view.findViewById<TextView>(R.id.productName).text = product.name
    view.findViewById<TextView>(R.id.productCategory).text =
        product.category
    view.findViewById<TextView>(R.id.productPrice).text =
        ProductRepository.formatPrice(product.price)
    ProductImages.show(
        context, product.image,
        view.findViewById<ImageView>(R.id.productImage)
    )
    return view
}
}
```

Adapter tái sử dụng `convertView` khi có thể, tránh tạo lại bố cục hàng mỗi lần cuộn. Ảnh được gán sau khi xác định sản phẩm ở vị trí hiện tại.

4.3 Lọc sản phẩm bằng Spinner

Danh mục được lấy từ chính dữ liệu bằng `map` và `distinct`. Ứng dụng thêm lựa chọn đầu tiên là "Tất cả". Khi người dùng chọn một mục, truy vấn cũ được xóa, `searchActive` trở về `false`, rồi danh sách được cập nhật theo danh mục.

Đoạn mã: Xử lý chọn danh mục trong `MainActivity.kt`

```
val categories = arrayListOf("Tất cả")
categories.addAll(allProducts.map { it.category }.distinct())

categorySpinner.onItemSelectedListener =
    object : AdapterView.OnItemSelectedListener {
        override fun onItemSelected(
            parent: AdapterView<*>?, view: View?,
            position: Int, id: Long
        ) {
            if (searchActive && position == 0) return
            searchActive = false
            searchInput.text.clear()
            val category = categories[position]
            showProducts(
```

```
                if (position == 0) allProducts
                else allProducts.filter { it.category == category }
            )
        }
        override fun onNothingSelected(parent: AdapterView<*>?) = Unit
    }
```

4.4 Tìm kiếm độc lập với bộ lọc

Khi nhấn nút tìm kiếm, ứng dụng chuẩn hóa chuỗi nhập, đưa Spinner về vị trí "Tất cả" và *luôn* lọc trên `allProducts`. Vì vậy, một danh mục đã chọn trước đó không giới hạn kết quả tìm kiếm. Tên và mã sản phẩm đều được xét. Hàm `normalize()` bỏ dấu kết hợp, xử lý riêng chữ `đ/Đ` và đổi về chữ thường; việc tìm theo tên không phân biệt dấu hoặc hoa thường.

Đoạn mã: Tìm kiếm trong `MainActivity.kt`

```
private fun performSearch() {
    val query = normalize(searchInput.text.toString().trim())
    searchActive = query.isNotEmpty()
    if (categorySpinner.selectedItemPosition != 0)
        categorySpinner.setSelection(0)
    val matches = if (query.isEmpty()) allProducts
        else allProducts.filter {
            normalize(it.name).contains(query) ||
                normalize(it.code).contains(query)
        }
    showProducts(matches)
}

private fun normalize(value: String): String =
    Normalizer.normalize(value, Normalizer.Form.NFD)
        .replace(Regex("\\p{M}+"), "")
        .replace('đ', 'd')
        .replace('Đ', 'D')
        .lowercase(Locale.ROOT)
```

Điều kiện `if (searchActive && position == 0) return` trong listener của Spinner ngăn lần cập nhật về "Tất cả" ghi đè kết quả tìm kiếm. Nếu ô tìm kiếm rỗng, `matches = allProducts` và giao diện trở về toàn bộ danh sách.

4.5 Mở Activity chi tiết

Sự kiện nhấn một hàng lấy đúng sản phẩm tại vị trí đã chọn và truyền `code` qua `Intent`. Activity thứ hai đọc lại JSON, tìm bản ghi có mã trùng khớp và điền dữ liệu vào các thành phần giao diện. Truyền mã thay vì truyền cả đối tượng giúp phần chi tiết luôn dựa trên nguồn JSON.

Đoạn mã: Điều hướng từ danh sách sang chi tiết

```
productList.onItemClickListener =
    AdapterView.OnItemClickListener { _, _, position, _ ->
        val product = productAdapter.getItem(position)
            ?: return@OnItemClickListener
        startActivity(
            Intent(this, ProductDetailActivity::class.java)
                .putExtra("product_code", product.code)
        )
    }
```

Đoạn mã: Đọc và hiển thị bản ghi trong `ProductDetailActivity.kt`

```
val code = intent.getStringExtra("product_code")
val product = try {
    ProductRepository.load(this).firstOrNull { it.code == code }
} catch (_: Exception) {
    null
}
if (product == null) {
    finish()
    return
}
findViewById<TextView>(R.id.detailName).text = product.name
ProductImages.show(
    this, product.image, findViewById<ImageView>(R.id.detailImage)
)
findViewById<TextView>(R.id.detailDescription).text =
    product.description
findViewById<TextView>(R.id.detailPrice).text =
    ProductRepository.formatPrice(product.price)
```

4.6 Đọc ảnh từ assets và hiển thị giao diện

Các URL ảnh có tiền tố `file:///android_asset/`. `ProductImages` loại bỏ tiền tố này để mở đúng tệp trong assets bằng `AssetManager`. Ảnh sau khi giải mã được lưu vào `LruCache` tối đa 8 MiB để hạn chế giải mã lặp lại khi cuộn danh sách.

Đoạn mã: Xử lý URL ảnh trong `ProductImages.kt`

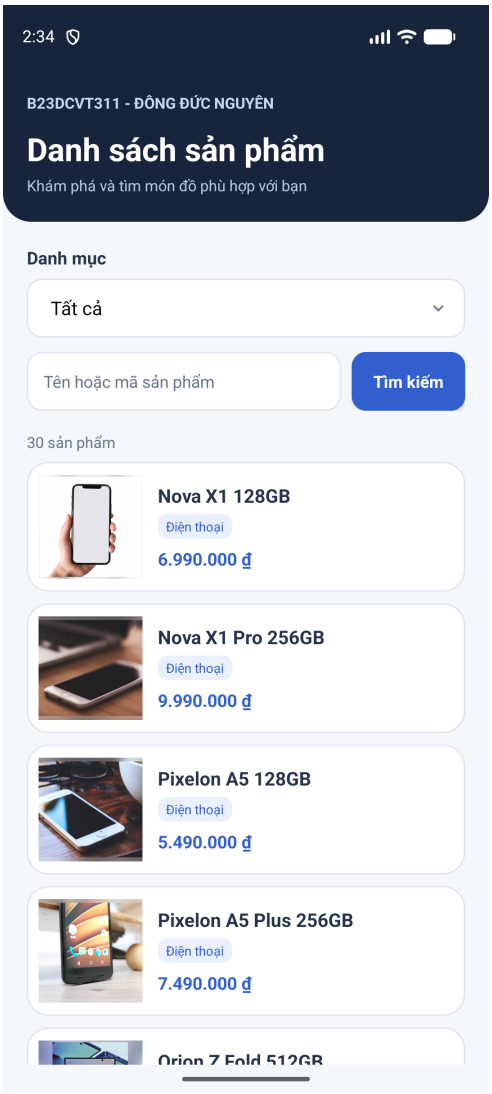
```
private const val ASSET_PREFIX = "file:///android_asset/"
private val cache = object : LruCache<String, Bitmap>(8 * 1024 * 1024) {
    override fun sizeOf(key: String, value: Bitmap): Int =
        value.byteCount
}
fun show(context: Context, imageUrl: String, view: ImageView) {
    val image = cache.get(imageUrl)
        ?: load(context, imageUrl)?.also { cache.put(imageUrl, it) }
    view.setImageBitmap(image)
}
```

Bố cục XML của `activity_main.xml` gồm phần đầu hiển thị thông tin sinh viên và tiêu đề, `Spinner`, `EditText`, `Button`, nhãn số kết quả và `ListView`. Bố cục `item_product.xml` quy định hình ảnh, tên, danh mục và giá của một hàng. Bố cục chi tiết có `ScrollView`, nhờ đó nội dung vẫn đọc được khi màn hình ngắn. Hai Activity được khai báo trong `AndroidManifest.xml`; chỉ `MainActivity` có `LAUNCHER`.

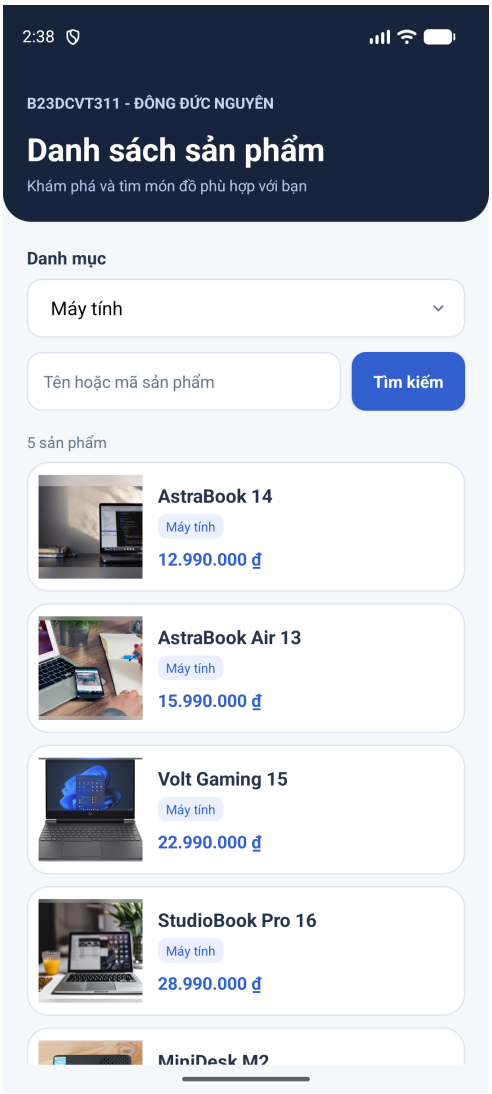
5 Ảnh chụp màn hình và đối chiếu chức năng

Các hình dưới đây là ảnh chụp thực tế của ứng dụng trên trình giả lập Android độ phân giải 1080×2424. Ảnh gốc nằm trong thư mục `screenshots/`; bản sao kèm báo cáo nằm trong `report/figures/`.

5.1 Danh sách ban đầu và lọc danh mục



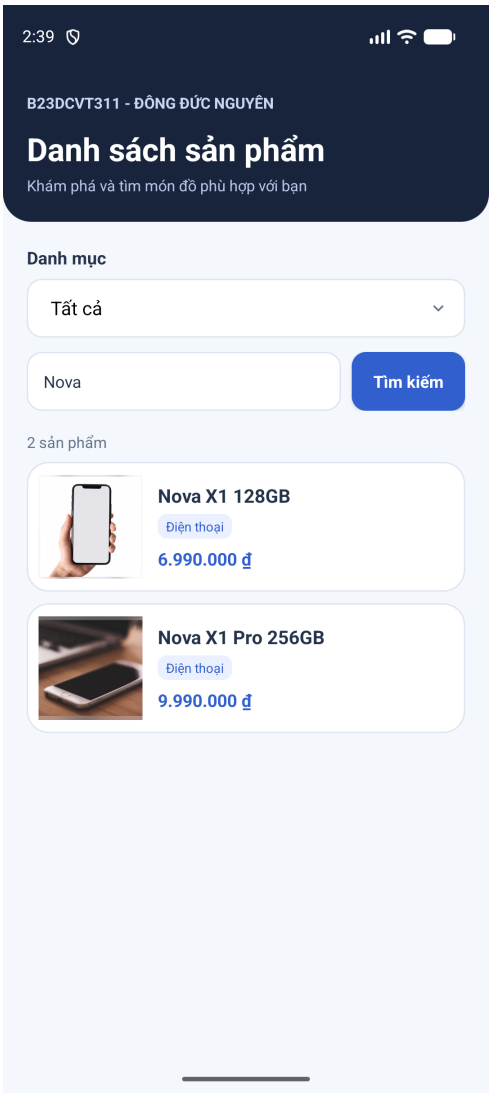
Hình 1: Màn hình ban đầu: "Tất cả", 30 sản phẩm.



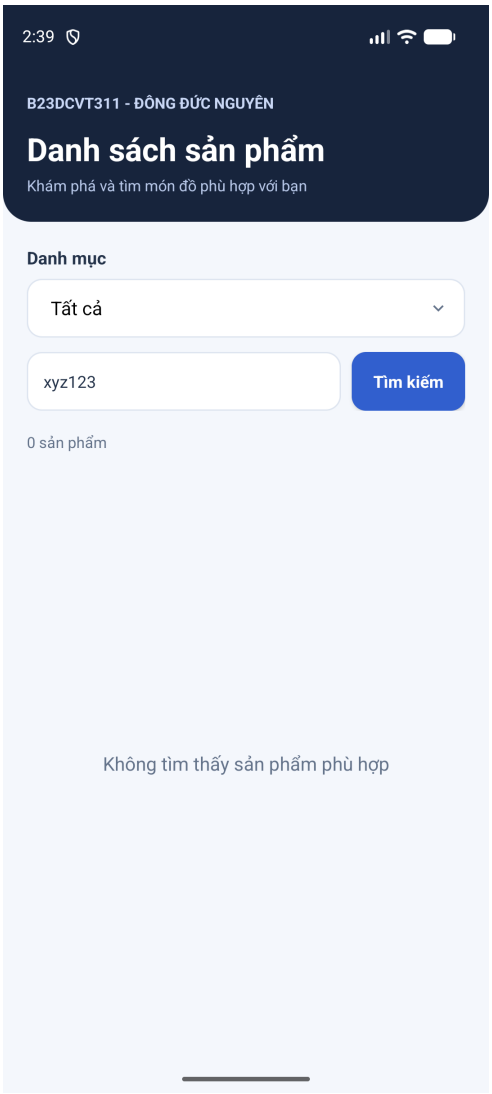
Hình 2: Chọn "Máy tính", còn 5 sản phẩm.

Hình 1 cho thấy phần đầu của Activity, Spinner, bộ tìm kiếm, nhãn tổng số 30 sản phẩm và các hàng có ảnh, tên, danh mục, giá. Hình 2 cho thấy Spinner đã chọn "Máy tính" và danh sách đúng 5 sản phẩm thuộc nhóm này.

5.2 Tìm kiếm có kết quả và không có kết quả



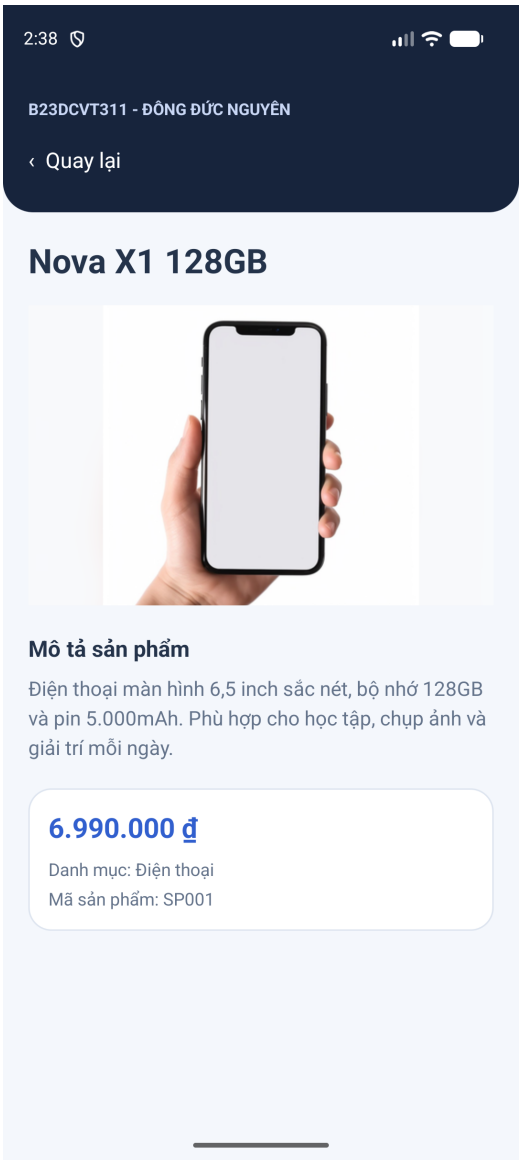
Hình 3: Tìm "Nova" trên toàn bộ sản phẩm: 2 kết quả.



Hình 4: Tìm "xyz123": 0 kết quả và thông báo rỗng.

Hình 3 thể hiện truy vấn "Nova" cho hai sản phẩm điện thoại; Spinner trở về "Tất cả", phù hợp yêu cầu tìm kiếm độc lập với danh mục. Hình 4 chứng minh trạng thái không có kết quả vẫn có phản hồi rõ ràng.

5.3 Màn hình chi tiết sản phẩm



Hình 5: Chi tiết sản phẩm `SP001`: tên, ảnh, mô tả và thông tin bổ sung.

Hình 5 cho thấy Activity thứ hai hiển thị sản phẩm "Nova X1 128GB", ảnh minh họa, mô tả, giá `6.990.000 đ`, danh mục "Điện thoại" và mã `SP001`. Phần đầu vẫn có thông tin sinh viên và nút "Quay lại".

6 Kiểm thử và đánh giá

6.1 Môi trường

Dự án dùng Kotlin, Android Gradle Plugin, `compileSdk = 35`, `minSdk = 23`, `targetSdk = 34` và JDK 17. Lệnh `assembleDebug` tạo được APK ở `app/build/outputs/apk/debug/`

`app-debug.apk`. Thiết bị giả lập đang dùng có độ phân giải 1080×2424.

6.2 Các trường hợp kiểm tra

Bảng 4: Kịch bản và kết quả quan sát

STT	Thao tác	Kết quả mong đợi và quan sát	Minh chứng
1	Mở ứng dụng	Hiện 30 sản phẩm; Spinner ở "Tất cả".	Hình 1
2	Chọn "Máy tính"	Hiện 5 sản phẩm của danh mục.	Hình 2
3	Tìm "Nova"	Spinner về "Tất cả"; hiện 2 kết quả.	Hình 3
4	Tìm "xyz123"	Hiện 0 sản phẩm và thông báo phù hợp.	Hình 4
5	Chọn `SP001`	Chi tiết có đúng tên, ảnh, mô tả và giá.	Hình 5

Kiểm tra dữ liệu cho thấy 30 bản ghi có mã riêng biệt, đủ sáu trường theo thiết kế và cả 30 đường dẫn ảnh đều trỏ đến tệp hiện có. Các trường hợp trong bảng đối chiếu hành vi giao diện với ảnh chụp thực tế.

6.3 Điểm đạt được và giới hạn

- Dữ liệu sản phẩm được đặt trong một tệp JSON; danh mục được suy ra từ dữ liệu, tránh nhập danh sách danh mục lần thứ hai trong mã Activity.
- Tìm kiếm dùng danh sách gốc, nên không chịu ảnh hưởng của lựa chọn Spinner trước đó. Giao diện có phản hồi khi kết quả rỗng.
- Ảnh được đóng gói cục bộ và có bộ đệm. Ứng dụng hoạt động ngoại tuyến.
- Vì JSON và ảnh là tệp trong APK, thay đổi dữ liệu hoặc ảnh yêu cầu tạo lại và cài lại ứng dụng. Tên mẫu sản phẩm trong bài là tên giả định; ảnh chụp CC0 đại diện đúng loại sản phẩm, không phải ảnh xác thực của một mẫu thương mại cùng tên. Thông tin nguồn và giấy phép được lưu trong metadata của từng PNG.

7 Kết luận

Bài thực hành đã triển khai đầy đủ chuỗi xử lý: đọc JSON $\rightarrow$ tạo `ArrayList<Product>` $\rightarrow$ hiển thị bằng `ArrayAdapter<Product>` và `ListView` $\rightarrow$ lọc và tìm kiếm $\rightarrow$ mở Activity chi tiết. Năm ảnh chụp màn hình minh chứng các trạng thái chính của ứng dụng. Cấu trúc mã tách phần dữ liệu, ảnh, adapter và giao diện, thuận tiện cho việc mở rộng thêm sản phẩm hoặc thay đổi bố cục về sau.

A Hướng dẫn biên dịch và mở báo cáo

A.1 Ứng dụng Android

Mở thư mục gốc bằng Android Studio và chạy cấu hình `app`, hoặc chạy `gradlew.bat :app:assembleDebug` trên Windows. APK debug được đặt tại `app/build/outputs/apk/debug/app-debug.apk`.

A.2 Báo cáo LaTeX

Biên dịch `report/bao_cao_san_pham.tex` bằng XeLaTeX hoặc Tectonic từ chính thư mục `report/`. Báo cáo sử dụng các ảnh trong `report/figures/`. Phông chính là Times New Roman; phông mã là Consolas. Các đoạn mã trong báo cáo tập trung vào những phần triển khai chức năng, còn mã nguồn đầy đủ nằm trong thư mục `app/src/main/`.